

1. Convert List<Employee> into TreeMap<Integer, Employee> keyed by id, keeping the employee with the higher salary when IDs clash.

Explanation: Use **Collectors.toMap**. The third argument, the **merge function**, handles the duplicate ID key clash. We define the merge function to compare the salaries of the two conflicting employees (e1 and e2) and return the one with the higher salary. We use **TreeMap::new** for the map factory to guarantee the map is a TreeMap (sorted by key).

```
List<Employee> employees = getEmployeesWithDuplicates();
Map<Integer, Employee> sortedEmployees = employees.stream()
    .collect(Collectors.toMap(
        Employee::getId, // Key Mapper: Use the Employee's ID
        Function.identity(), // Value Mapper: The Employee object itself
        (e1, e2) -> e1.getSalary() > e2.getSalary() ? e1 : e2, // Merge
        Function: Keep the Employee with the higher salary
        TreeMap::new // Map Factory: Ensure the result is a TreeMap
    ));
```

2. Generate a Map<String, Long> containing only product codes that occur more than once along with their occurrence counts.

Explanation: First, use **Collectors.groupingBy** to count the frequency of all product codes. Then, stream over the resulting map's **entrySet()** and use **filter()** to keep only entries where the count (getValue()) is greater than 1. Finally, collect the filtered entries into a new map.

```
List<String> codes = Arrays.asList("A", "B", "A", "C", "B", "A");
Map<String, Long> duplicateCounts = codes.stream()
    .collect(Collectors.groupingBy(Function.identity(),
        Collectors.counting())) // 1. Count all occurrences
    .entrySet().stream()
    .filter(entry -> entry.getValue() > 1) // 2. Filter: Keep only those
    counted > 1
    .collect(Collectors.toMap(Map.Entry::getKey, Map.Entry::getValue));
// 3. Collect into final Map
// Result: {A=3, B=2}
```

3. Find the single highest-paid employee and return it as an Optional<Employee>.

Explanation: Use the **max()** terminal operation, which returns an **Optional** for safety. We provide a Comparator that compares employees based on their salary field (Employee::getSalary).

```
List<Employee> employees = getEmployeeList();
Optional<Employee> highestPaid = employees.stream()
```

```
        .max(Comparator.comparing(Employee::getSalary)); // Find the max based on salary field
```

4. Produce a Map<Department, Long> showing how many employees are in each department.

Explanation: Use the **Collectors.groupingBy** collector. The grouping key is the Department object (extracted using `Employee::getDepartment`), and the downstream collector is **Collectors.counting()**, which counts the number of elements in each group.

```
List<Employee> employees = getEmployeeList();
Map<Department, Long> employeesByDept = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDepartment,
        Collectors.counting()));
```

5. Convert a List<String> into a single comma-separated string "java,spring,microservices".

Explanation: Use the **Collectors.joining** collector. This collector takes a delimiter argument (in this case, a comma followed by a space) and efficiently concatenates all elements of the stream into a single string.

```
List<String> tags = Arrays.asList("java", "spring", "microservices");
String result = tags.stream()
    .collect(Collectors.joining(",")); // Joins all elements with a comma delimiter
// Result: "java,spring,microservices"
```

6. Convert List<Order> into Map<Long, Order> using orderId as the key and handle duplicate orderIds.

Explanation: Use **Collectors.toMap**. The key mapper is `Order::getOrderId`. The merge function handles the duplicate ID clash by simply **keeping the first encountered object** (`(o1, o2) -> o1`).

```
List<Order> orders = getOrderList();
Map<Long, Order> orderMap = orders.stream()
    .collect(Collectors.toMap(
        Order::getOrderId, // Key Mapper: Order ID
        Function.identity(), // Value Mapper: The Order object itself
        (o1, o2) -> o1 // Merge Function: If duplicates occur, keep the first one (o1)
    ));
```

7. Return a new list of employees sorted by salary in descending order.

Explanation: Use the **sorted()** intermediate operation. Provide a Comparator that compares employees based on salary, and use **.reversed()** to change the sort order to descending.

```
List<Employee> employees = getEmployeeList();
List<Employee> sortedBySalary = employees.stream()
    .sorted(Comparator.comparing(Employee::getSalary).reversed())    //
    // Compare salary and reverse order
    .collect(Collectors.toList());
```

8. Return a list of email addresses that appear more than once.

Explanation: This is a slight variation of Q2. You group and count all emails, then filter the resulting map's entries to keep only those with a count greater than 1, extracting the key (the email address).

```
List<String> emails = Arrays.asList("a@a", "b@b", "a@a");
List<String> duplicateEmails = emails.stream()
    .collect(Collectors.groupingBy(Function.identity(),
    Collectors.counting()))
    .entrySet().stream()
    .filter(entry -> entry.getValue() > 1) // Keep entries where count >
1
    .map(Map.Entry::getKey) // Extract the email address
    .collect(Collectors.toList());
// Result: ["a@a"]
```

9. Remove duplicate customers based on email, keeping only one customer per unique email.

Explanation: Use **Collectors.toMap**. The key is the email address. Since emails might clash, the merge function is necessary. We simply define the merge function to keep the first customer ((c1, c2) -> c1) encountered for a given email. Then, we collect only the map's values.

```
List<Customer> customers = getCustomerList();
List<Customer> uniqueCustomers = customers.stream()
    .collect(Collectors.toMap(
        Customer::getEmail,
        Function.identity(),
        (c1, c2) -> c1 // Keep the first customer object found
    ))
    .values().stream() // Get the list of unique Customer objects (the
    values)
    .collect(Collectors.toList());
```

10. Group words into anagram groups, returning a `Map<String, List<String>>` where the key is the sorted character sequence.

Explanation: Use `Collectors.groupingBy`. The key mapping function converts the word into a sorted character array and then back into a canonical string (e.g., "listen" -> "eilnst"). All original words that share the same canonical key are grouped into a single list.

```
List<String> words = Arrays.asList("listen", "silent", "car", "arc");
Map<String, List<String>> anagramGroups = words.stream()
    .collect(Collectors.groupingBy(word -> {
        char[] chars = word.toCharArray();
        Arrays.sort(chars);
        return new String(chars); // Canonical Key: e.g., "eilnst"
    }));
// Result: {"eilnst": [listen, silent], "acr": [car, arc]}
```

11. Find the employee who joined earliest.

Explanation: Use the `min()` terminal operation, which returns an `Optional`. Provide a `Comparator` that compares employees based on their `joiningDate` field.

```
List<Employee> employees = getEmployeeList();
Optional<Employee> earliestJoiner = employees.stream()
    .min(Comparator.comparing(Employee::getJoiningDate)); // Find min
// based on LocalDate field
```

12. Convert a `Map<String, Integer>` into a `List<StockInfo>`.

Explanation: Stream over the map's `entrySet()`. Use the `map()` operation to transform each `Map.Entry` into a new `StockInfo` object, passing the key (product code) and value (quantity) to the constructor.

```
Map<String, Integer> stockMap = Map.of("APPLE", 50, "ORANGE", 100);
List<StockInfo> stockList = stockMap.entrySet().stream()
    .map(entry -> new StockInfo(entry.getKey(), entry.getValue())) // Map
    .collect(Collectors.toList());
```

13. Compute the median transaction amount (assuming you can load all of them in memory and sort).

Explanation: The median requires sorting. Sort the amounts and find the element at the middle index (or average of two middle elements for an even count).

```
List<Double> amounts = Arrays.asList(10.0, 50.0, 20.0, 40.0, 30.0);
```

```
List<Double> sorted =  
amounts.stream().sorted().collect(Collectors.toList());  
int size = sorted.size();  
  
double median = (size % 2 != 0)  
    ? sorted.get(size / 2) // Odd size: middle element  
    : (sorted.get(size / 2 - 1) + sorted.get(size / 2)) / 2.0; // Even  
size: average of two middle elements
```

14. Convert a List<User> into a Map<Long, User> keyed by userId.

Explanation: Use **Collectors.toMap**. The key mapper is User::getUserId. Since each user has a unique ID, the merge function argument is optional (or you can use (u1, u2) -> u1 for safety).

```
List<User> users = getUserList();  
Map<Long, User> userMap = users.stream()  
    .collect(Collectors.toMap(User::getUserId, Function.identity())); //  
Key: userId, Value: User object
```

15. Convert List<String> in "key,value" format into a Map<String, Integer>.

Explanation: Use **Collectors.toMap**. The key mapper splits the string and takes the first part (index 0). The value mapper splits the string and converts the second part (index 1) into an integer.

```
List<String> data = Arrays.asList("apple,3", "banana,10", "cherry,5");  
Map<String, Integer> fruitMap = data.stream()  
    .map(s -> s.split(",")) // Split into String[] {"key", "value"}  
    .collect(Collectors.toMap(  
        arr -> arr[0], // Key: arr[0] (fruit name)  
        arr -> Integer.parseInt(arr[1]) // Value: arr[1] converted to  
        Integer  
    ));  
// Result: {apple=3, banana=10, cherry=5}
```